

12. Composite Design Pattern in Java

12.1 Introduction

The **Composite Design Pattern** is a **structural pattern** used to treat individual objects and compositions of objects uniformly.

12.2 Problem it Solves

When working with tree-like structures, managing objects and groups of objects differently creates complexity. Composite solves this by treating all elements uniformly.

12.3 What is Decorator Pattern?

It allows you to compose objects into tree structures and work with these structures as if they were individual objects.

12.4 Real-World Analogy

A directory (folder) can contain files or other directories. You can perform operations like size or delete uniformly on both files and folders.

12.5 Structure

```
package com.mannemlokes;

import java.util.ArrayList;
import java.util.List;

interface Component {
    void showDetails();
}

class Leaf implements Component {
    public void showDetails() {
        System.out.println("Leaf operation");
    }
}

class Composite implements Component {
    private List<Component> children = new ArrayList<>();

    public void add(Component component) {
        children.add(component);
    }

    public void showDetails() {
        for (Component child : children) {
            child.showDetails();
        }
    }
}
```

12.6 Benefits

- Treats single and group objects uniformly

- Simplifies client code
- Flexible tree structure

12.7 Implementation Steps

1. Define a component interface
2. Implement leaf nodes
3. Implement composite nodes with child management
4. Use component interface for all interaction

12.8 Examples

Example1: File System (File & Folder)

```
package com.mannemlokes;

import java.util.ArrayList;
import java.util.List;

interface FileSystem {
    void display(String indent);
}

class File implements FileSystem {
    private String name;

    public File(String name) {
        this.name = name;
    }

    public void display(String indent) {
        System.out.println(indent + "File: " + name);
    }
}

class Folder implements FileSystem {
    private String name;
    private List<FileSystem> contents = new ArrayList<>();

    public Folder(String name) {
        this.name = name;
    }

    public void add(FileSystem fileSystem) {
        contents.add(fileSystem);
    }

    public void display(String indent) {
        System.out.println(indent + "Folder: " + name);
        for (FileSystem item : contents) {
            item.display(indent + " ");
        }
    }
}

public class Example1FileExplorer {
    public static void main(String[] args) {
        File file1 = new File("Resume.pdf");
```

```
        File file2 = new File("Budget.xlsx");

        Folder documents = new Folder("Documents");
        documents.add(file1);
        documents.add(file2);

        Folder root = new Folder("Root");
        root.add(documents);
        root.add(new File("Readme.txt"));

        root.display("");
    }
}
```

Output:

```
Folder: Root
  Folder: Documents
    File: Resume.pdf
    File: Budget.xlsx
  File: Readme.txt
```

Example2: Employee Hierarchy

```
package com.mannemlokes;

import java.util.ArrayList;
import java.util.List;

interface Employee {
    void showDetails();
}

class Developer implements Employee {
    private String name;

    public Developer(String name) {
        this.name = name;
    }

    public void showDetails() {
        System.out.println("Developer: " + name);
    }
}

class Manager implements Employee {
    private String name;
    private List<Employee> team = new ArrayList<>();

    public Manager(String name) {
        this.name = name;
    }

    public void add(Employee employee) {
        team.add(employee);
    }

    public void showDetails() {
```

```

        System.out.println("Manager: " + name);
        for (Employee emp : team) {
            emp.showDetails();
        }
    }
}

public class Example2Company {
    public static void main(String[] args) {
        Developer dev1 = new Developer("Alice");
        Developer dev2 = new Developer("Bob");

        Manager manager = new Manager("Charles");
        manager.add(dev1);
        manager.add(dev2);

        manager.showDetails();
    }
}

```

Output:

```

Manager: Charles
Developer: Alice
Developer: Bob

```

Example3: GUI Components

```

package com.mannemlokesk;

import java.util.ArrayList;
import java.util.List;

interface GUIComponent {
    void render();
}

class Button implements GUIComponent {
    private String label;

    public Button(String label) {
        this.label = label;
    }

    public void render() {
        System.out.println("Rendering Button: " + label);
    }
}

class Panel implements GUIComponent {
    private List<GUIComponent> components = new ArrayList<>();

    public void add(GUIComponent component) {
        components.add(component);
    }

    public void render() {
        System.out.println("Rendering Panel");
    }
}

```

```

        for (GUIComponent c : components) {
            c.render();
        }
    }
}

public class Example3GUIApp {
    public static void main(String[] args) {
        Button btn1 = new Button("Submit");
        Button btn2 = new Button("Cancel");

        Panel panel = new Panel();
        panel.add(btn1);
        panel.add(btn2);

        panel.render();
    }
}

```

Output:

```

Rendering Panel
Rendering Button: Submit
Rendering Button: Cancel

```

12.9 Composite vs Other Patterns

Pattern	Purpose
Composite	Treat individual and groups uniformly
Decorator	Add responsibilities to an object
Strategy	Change behaviour dynamically
Builder	Step-by-step object construction

12.10 Pros and Cons

**Pros**

- Uniform treatment of leaf and composite
- Flexible and scalable structure

**Cons**

- Can make system overly general
- Might be hard to restrict certain behaviours

12.11 When to Use / Not to Use

Use When:

- You have a tree structure (menu, org chart, file system)
- You want uniform operations on individual and groups

Avoid When:

- You don't need to treat objects uniformly
- You're not working with hierarchies

12.12 Summary

Feature	Description
Intent	Treat individual and composite objects
Structure	Component → Leaf + Composite
Key Benefit	Simplifies tree-like structure management

The **Composite Pattern** is ideal for building recursive structures like menus, GUIs, and file systems.